

API Gateway Performance Debugging

Complete guide to identify and fix slow API Gateway responses.

Step 1: Measure and Identify

Add Request Timing

```
import time
from fastapi import FastAPI, Request
import logging

app = FastAPI()

@app.middleware("http")
async def log_request_time(request: Request, call_next):
    start = time.time()

    response = await call_next(request)

    duration = time.time() - start

    # Log slow requests
    if duration > 1.0: # > 1 second
        logging.warning(
            f"SLOW REQUEST: {request.method} {request.url.path} "
            f"took {duration:.2f}s"
        )

    response.headers["X-Process-Time"] = str(duration)
    return response
```

Distributed Tracing

Using OpenTelemetry:

```
from opentelemetry import trace
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.requests import RequestsInstrumentor
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import ConsoleSpanExporter, BatchSpanProcessor

# Setup tracing
trace.set_tracer_provider(TracerProvider())
trace.get_tracer_provider().add_span_processor(
    BatchSpanProcessor(ConsoleSpanExporter())
)
```

```
# Auto-instrument
FastAPIInstrumentor.instrument_app(app)
RequestsInstrumentor().instrument()

@app.get("/user/{user_id}/dashboard")
async def get_dashboard(user_id: int):
    tracer = trace.get_tracer(__name__)

    with tracer.start_as_current_span("fetch-user"):
        user = await fetch_user(user_id) # Trace this call

    with tracer.start_as_current_span("fetch-orders"):
        orders = await fetch_orders(user_id) # Trace this call

    with tracer.start_as_current_span("fetch-recommendations"):
        recs = await fetch_recommendations(user_id) # Trace this call

    return {"user": user, "orders": orders, "recommendations": recs}
```

Output shows:

```
GET /user/123/dashboard - 3.5s
├─ fetch-user - 0.5s
├─ fetch-orders - 2.8s ← BOTTLENECK!
└─ fetch-recommendations - 0.2s
```

Manual Instrumentation

```
import time
from functools import wraps

def time_it(func):
    @wraps(func)
    async def wrapper(*args, **kwargs):
        start = time.time()
        result = await func(*args, **kwargs)
        duration = time.time() - start
        print(f"{func.__name__} took {duration:.3f}s")
        return result
    return wrapper

@time_it
async def fetch_user(user_id):
    return await http_client.get(f"http://user-service/users/{user_id}")

@time_it
async def fetch_orders(user_id):
    return await http_client.get(f"http://order-service/orders?user={user_id}")
```

```
@time_it
async def fetch_recommendations(user_id):
    return await http_client.get(f"http://rec-service/recommend/{user_id}")
```

Step 2: Common Causes and Fixes

1. Sequential Service Calls (Most Common!)

Problem:

```
# BAD: Sequential calls (3s + 2s + 1s = 6s total)
async def get_dashboard(user_id: int):
    user = await fetch_user(user_id)          # 3s
    orders = await fetch_orders(user_id)      # 2s
    recs = await fetch_recommendations(user_id) # 1s
    return {"user": user, "orders": orders, "recs": recs}
```

Fix: Parallel Calls

```
import asyncio

# GOOD: Parallel calls (max(3s, 2s, 1s) = 3s total)
async def get_dashboard(user_id: int):
    user_task = fetch_user(user_id)
    orders_task = fetch_orders(user_id)
    recs_task = fetch_recommendations(user_id)

    # Run all in parallel
    user, orders, recs = await asyncio.gather(
        user_task,
        orders_task,
        recs_task
    )

    return {"user": user, "orders": orders, "recs": recs}
```

Impact: 6s → 3s (50% faster)

2. Slow Downstream Service

Identify:

```
# Check service response times
curl -w "@curl-format.txt" -o /dev/null -s http://order-service/orders?user=123

# curl-format.txt:
time_namelookup: %{time_namelookup}s
```

```
time_connect:      %{time_connect}s
time_starttransfer: %{time_starttransfer}s
time_total:        %{time_total}s
```

Fix 1: Add Timeout

```
import httpx

async with httpx.AsyncClient(timeout=2.0) as client:
    try:
        response = await client.get(
            f"http://slow-service/api",
            timeout=2.0 # Fail fast
        )
    except httpx.TimeoutException:
        return fallback_response()
```

Fix 2: Optimize Downstream Service

- Add database indexes
- Add caching
- Reduce payload size
- Optimize queries

Fix 3: Cache at Gateway

```
from cachetools import TTLCache
import asyncio

cache = TTLCache(maxsize=1000, ttl=300) # 5 min TTL

async def fetch_user_cached(user_id: int):
    cache_key = f"user:{user_id}"

    if cache_key in cache:
        return cache[cache_key]

    user = await fetch_user(user_id)
    cache[cache_key] = user
    return user
```

3. Network Latency

Identify:

```
import time
import httpx
```

```
start = time.time()
async with httpx.AsyncClient() as client:
    # DNS lookup
    dns_start = time.time()
    response = await client.get("http://service-name/api")
    dns_time = time.time() - dns_start

print(f"DNS + Connect + Transfer: {dns_time:.3f}s")
```

Fix 1: Connection Pooling

```
# BAD: New connection per request
async def call_service():
    async with httpx.AsyncClient() as client:
        return await client.get("http://service/api")

# GOOD: Reuse connections
client = httpx.AsyncClient(
    limits=httpx.Limits(
        max_keepalive_connections=20,
        max_connections=100
    )
)

async def call_service():
    return await client.get("http://service/api")
```

Fix 2: Use Service Mesh / Internal Network

```
# Kubernetes: Use cluster DNS
http://order-service.default.svc.cluster.local/orders

# Instead of external DNS
http://order-service.example.com/orders
```

4. Large Response Payloads

Identify:

```
response = await client.get("http://service/api")
size_kb = len(response.content) / 1024
print(f"Response size: {size_kb:.2f} KB")
```

Fix 1: Pagination

```
# BAD: Return all 10,000 orders
@app.get("/orders")
def get_orders(user_id: int):
    return Order.query.filter_by(user_id=user_id).all()

# GOOD: Paginate
@app.get("/orders")
def get_orders(user_id: int, page: int = 1, limit: int = 20):
    offset = (page - 1) * limit
    orders = Order.query.filter_by(user_id=user_id)\
        .limit(limit)\
        .offset(offset)\
        .all()
    return orders
```

Fix 2: Field Selection

```
# Client specifies fields
@app.get("/users/{user_id}")
def get_user(user_id: int, fields: str = None):
    user = User.query.get(user_id)

    if fields:
        # Return only requested fields
        field_list = fields.split(',')
        return {k: v for k, v in user.__dict__.items() if k in field_list}

    return user
```

Fix 3: Compression

```
from fastapi.middleware.gzip import GZipMiddleware

app.add_middleware(GZipMiddleware, minimum_size=1000)
```

5. Gateway Overload (Too Many Requests)

Identify:

```
from prometheus_client import Histogram, Counter

REQUEST_LATENCY = Histogram(
    'gateway_request_duration_seconds',
    'Request latency'
)
ACTIVE_REQUESTS = Counter('gateway_active_requests', 'Active requests')
```

```
@app.middleware("http")
async def monitor_requests(request: Request, call_next):
    ACTIVE_REQUESTS.inc()

    with REQUEST_LATENCY.time():
        response = await call_next(request)

    ACTIVE_REQUESTS.dec()
    return response
```

Fix 1: Rate Limiting

```
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)

@app.get("/api/orders")
@limiter.limit("100/minute")
async def get_orders():
    pass
```

Fix 2: Load Balancing

```
# Deploy multiple gateway instances
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-gateway
spec:
  replicas: 3 # 3 instances
  selector:
    matchLabels:
      app: api-gateway
```

Fix 3: Circuit Breaker

```
from pybreaker import CircuitBreaker

breaker = CircuitBreaker(fail_max=5, timeout_duration=60)

@breaker
async def call_service():
    return await client.get("http://service/api")

try:
```

```
    result = await call_service()
except CircuitBreakerError:
    return {"error": "Service unavailable", "fallback": True}
```

6. Synchronous Blocking Code

Problem:

```
# BAD: Blocking I/O in async function
async def get_data(user_id: int):
    # This blocks the event loop!
    import requests
    response = requests.get(f"http://service/users/{user_id}")
    return response.json()
```

Fix: Use Async Libraries

```
# GOOD: Non-blocking I/O
import httpx

async def get_data(user_id: int):
    async with httpx.AsyncClient() as client:
        response = await client.get(f"http://service/users/{user_id}")
        return response.json()
```

7. Database Query at Gateway

Problem:

```
# BAD: Gateway queries database
@app.get("/orders/{order_id}")
async def get_order(order_id: int):
    order = await db.execute("SELECT * FROM orders WHERE id = ?", order_id)
    return order
```

Fix: Gateway Should Proxy, Not Query

```
# GOOD: Forward to service
@app.get("/orders/{order_id}")
async def get_order(order_id: int):
    response = await http_client.get(f"http://order-service/orders/{order_id}")
    return response.json()
```


8. Authentication/Authorization Overhead

Problem:

```
# BAD: Validate JWT on every request
@app.middleware("http")
async def auth_middleware(request: Request, call_next):
    token = request.headers.get("Authorization")
    user = verify_jwt(token) # Expensive crypto operation
    request.state.user = user
    return await call_next(request)
```

Fix: Cache Auth Results

```
from cachetools import TTLCache

auth_cache = TTLCache(maxsize=10000, ttl=300) # 5 min

@app.middleware("http")
async def auth_middleware(request: Request, call_next):
    token = request.headers.get("Authorization")

    if token in auth_cache:
        request.state.user = auth_cache[token]
    else:
        user = verify_jwt(token)
        auth_cache[token] = user
        request.state.user = user

    return await call_next(request)
```

Step 3: Monitoring and Alerting

Prometheus Metrics

```
from prometheus_client import Histogram, Counter, Gauge
from prometheus_fastapi_instrumentator import Instrumentator

app = FastAPI()

# Auto-instrument
Instrumentator().instrument(app).expose(app)

# Custom metrics
REQUEST_DURATION = Histogram(
    'gateway_request_duration_seconds',
    'Request duration',
    ['method', 'endpoint']
```

```

)

DOWNSTREAM_DURATION = Histogram(
    'gateway_downstream_duration_seconds',
    'Downstream service duration',
    ['service']
)

@app.get("/orders")
async def get_orders():
    with REQUEST_DURATION.labels(method='GET', endpoint='/orders').time():
        with DOWNSTREAM_DURATION.labels(service='order-service').time():
            orders = await fetch_orders()
            return orders

```

Alerts

```

# Prometheus alert rules
groups:
- name: gateway
  rules:
  - alert: GatewayHighLatency
    expr: histogram_quantile(0.95, gateway_request_duration_seconds) > 2
    for: 5m
    annotations:
      summary: "Gateway 95th percentile latency > 2s"

  - alert: DownstreamServiceSlow
    expr: histogram_quantile(0.95, gateway_downstream_duration_seconds) > 1
    for: 5m
    labels:
      severity: warning

```

Step 4: Load Testing

Identify Bottleneck Under Load

```

# locustfile.py
from locust import HttpUser, task, between

class GatewayUser(HttpUser):
    wait_time = between(1, 3)

    @task
    def get_dashboard(self):
        self.client.get("/user/123/dashboard")

    @task(3) # 3x more frequent

```

```
def get_orders(self):
    self.client.get("/orders?user=123")
```

Run:

```
locust -f locustfile.py --host=http://localhost:8080
# Open http://localhost:8089
# Start with 10 users, ramp up to 100
```

Analyze Results:

- Response times increase linearly → Good scaling
- Response times spike at X users → Bottleneck at X
- Errors increase → Circuit breaker or timeout issues

Debugging Checklist

```
# Add this comprehensive logging
import time
import logging

@app.middleware("http")
async def debug_middleware(request: Request, call_next):
    start = time.time()

    # Log request
    logging.info(f"> {request.method} {request.url.path}")

    # Track stages
    timings = {}

    # Authentication
    auth_start = time.time()
    # ... auth code ...
    timings['auth'] = time.time() - auth_start

    # Process request
    response = await call_next(request)

    # Total time
    total = time.time() - start
    timings['total'] = total

    # Log results
    if total > 1.0:
        logging.warning(
            f"SLOW: {request.method} {request.url.path} "
            f"total={total:.3f}s auth={timings['auth']:.3f}s"
        )
```

```
# Add to response headers
response.headers["X-Timing-Total"] = f"{total:.3f}"
response.headers["X-Timing-Auth"] = f"{timings['auth']:.3f}"

return response
```

Quick Wins Summary

Immediate (No Code Change):

1. Scale gateway horizontally (add replicas)
2. Add load balancer
3. Enable compression

Easy (< 1 day):

1. Make service calls parallel
2. Add timeouts
3. Add connection pooling
4. Add response caching

Medium (1-3 days):

1. Add distributed tracing
2. Implement circuit breaker
3. Add rate limiting
4. Optimize slow downstream services

Hard (> 1 week):

1. Implement service mesh
2. Refactor to event-driven
3. Add CDN for static content
4. Database optimization

Real-World Example

Before (6s response time):

```
@app.get("/user/{user_id}/dashboard")
async def get_dashboard(user_id: int):
    # Sequential calls
    user = await fetch_user(user_id)           # 2s
    orders = await fetch_orders(user_id)       # 3s
    recs = await fetch_recommendations(user_id) # 1s

    return {
        "user": user,
        "orders": orders,
```

```
    "recommendations": recs
}
```

After (2s response time):

```
from cachetools import TTLCache
import asyncio

cache = TTLCache(maxsize=1000, ttl=300)

@app.get("/user/{user_id}/dashboard")
async def get_dashboard(user_id: int):
    # Check cache first
    cache_key = f"dashboard:{user_id}"
    if cache_key in cache:
        return cache[cache_key]

    # Parallel calls with timeout
    async with httpx.AsyncClient(timeout=2.0) as client:
        user, orders, recs = await asyncio.gather(
            fetch_user(user_id, client),
            fetch_orders(user_id, client, limit=10), # Paginate
            fetch_recommendations(user_id, client),
            return_exceptions=True # Don't fail if one fails
        )

    # Build response
    result = {
        "user": user if not isinstance(user, Exception) else None,
        "orders": orders if not isinstance(orders, Exception) else [],
        "recommendations": recs if not isinstance(recs, Exception) else []
    }

    # Cache for 5 minutes
    cache[cache_key] = result

    return result
```

Improvements:

- Sequential → Parallel: 6s → 2s (67% faster)
- Added caching: 2s → ~0ms for cached
- Added pagination: Smaller payload
- Added timeout: Fail fast
- Graceful degradation: Partial response if service fails

Interview Answer Template

"API Gateway is slow, how do you debug?"

1. **Measure first:** Add timing middleware, use distributed tracing
2. **Identify bottleneck:** Which service is slow? Network? Gateway itself?
3. **Common causes:**
 - Sequential calls → Make parallel
 - Slow downstream → Add timeout, cache, optimize
 - Large payload → Paginate, compress
 - Network latency → Connection pooling, internal network
4. **Monitor:** Prometheus metrics, alerts
5. **Test:** Load test to verify fix
6. **Prevent:** Circuit breaker, rate limiting, autoscaling